Strategic Optimization: Analyzing and Comparing Algorithms for Optimal Decision-Making

GROUP 10

Bousmar Cyril - 63401800
UCLouvain, EPL - SINF
cyril.bousmar@uclouvain.be

Delsart Mathis - 31302100
UCLouvain, EPL - INFO
mathis.delsart@student.uclouvain.be

Lamien Sienou - 72642301
UCLouvain, EPL - SINF
sienou.lamien@student.uclouvain.be

I. Introduction

Markov Decision Processes (MDPs) are fundamental in decision-making under uncertainty, finding applications in artificial intelligence, robotics, and game theory. In this technical report, we explore the application of *MDPs* to the *Snakes and Ladders* game, where the objective is to determine the optimal die selection strategy that minimizes the number of turns required to reach the goal. We use the *Value Iteration* algorithm to compute optimal policies and compare its performance against various suboptimal strategies, including purely random and deterministic dice selections. Additionally, we investigate the *Q-learning* algorithm as an alternative reinforcement learning approach, which we analyze and compare to *Value Iteration*.

The report is structured as follows: Section II introduces the game and its notations. Section III outlines the theoretical foundations of *MDPs* and *Q-learning*. Section IV details the implementation aspects, including function design and computational considerations. Section V explains the evaluation workflow across different layouts and strategies. Section VI presents and analyses experimental results, and Section VII concludes with key findings and possible extensions.

II. Snakes and Ladders Game

The objective of the game is to reach the final cell, starting from the first one. At each turn, the player selects one die from the three available. The result determines how many cells the player advances. Figure 1 illustrates an example of such a board layout.

Two variants of the game are considered: (1) the player must land exactly on the final cell to win, otherwise they *cycle back* to the starting position; and (2) the player wins upon reaching or surpassing the final cell.

The three die are defined as follows:

- *Security Die*: Moves 0 or 1 cell (each with a probability of 0.5); immune to cell effects.
- *Normal Die*: Moves 0, 1, or 2 cells (each with a probability of 1/3); triggers cell effects with a probability of 0.5.
- *Risky Die*: Moves 0 to 3 cells (each with a probability of 0.25); always triggers cell effects.

The board consists of different types of cells, each contributing to a specific layout: *normal*, *prison*, *penalty*, *restart*, and *bonus*. Landing on a cell may trigger its effect, with a probability depending on the chosen die. Effects are respectively: *no effect*, the player *loses one turn*, the player *moves backward by 3*, the player *returns to the first cell*, and the player *plays again*.

For convenience, we will refer visually to a layout as in Figure 1, where *normal* cells are squares, *prisons* are circles, *penalties* are diamonds, *restarts* are 12-pointed stars, and *bonuses* are 6-pointed stars.

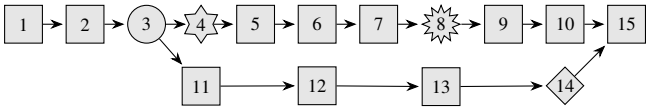

Fig. 1: Basic layout example: all cells are *normal* except for 3 (*prison*), 4 (*bonus*), 8 (*restart*), 14 (*penalty*). The starting cell is 1, and the goal cell is 15.

By its nature, the *Snakes and Ladders* game is stochastic, and the outcome of one action do not depend on previously taken ones. This meets the requirements to use *Markov Decision Process* to find the most effective course of action (i.e. the optimal policy).

III. Theoretical Background

For consistency and formalism, the main symbols used throughout this report are described in Table I.

TABLE I: Notations

Notation	Description
$\mathcal{A}$	Action space := set of available dice in the game.
$\mathcal{S}$	State space := the set of all possible board positions.
$C(a\|k)$	Cost of action $a \in \mathcal{A}$, in state $k \in \mathcal{S}$. Shortened by c_k.
ϕ_k	Penalty in state $k \in \mathcal{S}$ (i.e. triggered effect of cells).
$P(k'\|k,a)$	Transition probability from state k to $k' \in \mathcal{S}$, through action $a \in \mathcal{A}$.
$\hat{\pi}$	Function that provides the set of optimal policies, $\forall k \in \mathcal{S}$.
$\hat{V}$	Set of cost of optimal policy, $\forall k \in \mathcal{S}$ (i.e. expected number of turns to reach goal state from k).

In *Snakes and Ladders*, taking an action always incurs the same cost; therefore, we define $C(a|k) = 1$. While penalties may arise due to cell effects, they are represented separately by ϕ_k (i.e. $+1$ for *prison*, -1 for *bonus*, 0 otherwise) for clarity.

A. Markov Decision Process (MDP)

Formally, a *Markov Decision Process (MDP)* is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, C, \gamma)$ [1], [2], where γ (set to 1 because of the episodic settings) is the discount factor, P is short for $P(k'|k,a)$, and C is short for $C(a|k)$.

The objective of the *MDP* is to find the optimal policy $\hat{\pi} : \mathcal{S} \rightarrow \mathcal{A}$ and the corresponding optimal cost function $\hat{V}_k$, ensuring the minimization of the expected cumulative cost required to reach the goal state from any other state.

B. Value-Iteration

MDP is solved using *Value Iteration* [1], [2], a dynamic programming method that iteratively computes $\hat{V}_k$ to determine the optimal policy $\hat{\pi}$. We apply the *Bellman optimality equation*, incorporating penalties ϕ_k:

$$\hat{V}_1 \leftarrow 0, \quad \text{(first state is } k = 1)$$

$$\hat{V}_k \leftarrow \min_{a \in \mathcal{A}_k} \left[C(a \mid k) + \sum_{k'} \phi_{k'} + P(k' \mid k, a) \cdot \hat{V}_{k'} \right] \quad (1)$$

The optimal action (i.e., the best dice choice) in state k is then given by:

$$\hat{V}_k^* = \arg\min_{a \in \mathcal{A}_k} \hat{V}_k$$

C. Q-learning

Another approach is to use a model-free method that finds the optimal policy through trial and errors [3], [4]. By leveraging the *Bellman equations*, we can implement a passive reinforcement learning algorithm based on the *Temporal-Difference (TD) equation*:

$$\hat{Q}(k,a) \leftarrow \hat{Q}(k,a) + \alpha_t \left(r_k + \gamma \max_{a'} \hat{Q}(k',a') - \hat{Q}(k,a) \right) \quad (2)$$

where r_k is the immediate reward of chosen action (see Algorithm 2, line 8), and actions are selected according to:

$$a = \begin{cases} \text{rand}(\mathcal{D}), & \text{with probability } \varepsilon_t \\ \arg\max_{a'} \hat{Q}(k,a'), & \text{otherwise} \end{cases} \quad (3)$$

This defines an *ε-greedy* algorithm, which balances *exploration* (i.e. selecting an action randomly with probability ε_t, the value of ε at time t) and *exploitation* (i.e. choosing the best-known action) balanced by α_t.

By simulating multiple episodes (i.e. games) starting from each state, the *Q-value* of actions are iteratively updated. Once training is complete, the value function and the optimal policy are extracted as:

$$\hat{V}_k = \min_a Q(k,a), \quad \hat{\pi}_k = \arg\min_a Q(k,a)$$

IV. Implementation

A. Value Iteration MDP

As an iterative algorithm, *Value Iteration* requires safeguards to prevent excessive runtime. We impose a maximum number of iterations, denoted as N_{max} and introduce an early stopping criterion based on a tolerance parameter ε.

The pseudo-code in Algorithm 1 outlines the procedure for computing and returning the value function $\hat{V}$ and the optimal policy $\hat{\pi}$ for all states. The algorithm starts from an initial configuration, assuming a given game layout and accounting for whether the game allows cycling or not.

Algorithm 1 MDP

Require: $layout$, $circle$
Ensure: List of $\hat{V}_k$ and $\hat{\pi}_k$, $\forall k \in \mathcal{S}$

```
N ← 0, N_max ← 500, ε ← 10^-9, δ ← +∞
∀k ∈ S : V̂_k ← 0 and π̂ ← 0
while δ > ε and N < N_max do
    δ ← 0, V̂_new ← V̂_k
    for k ∈ S | k not final do
        for a ∈ D do
            T_k ← {(P(k'|t), φ_k') | t = (k, a)}
            V_k[a] ← c_k + Σ_{t∈T_k} (φ_k' + P(k'|t) · V̂_k[k'])
        end for
        π̂[k] ← arg min_{a∈V_k} V_k[a], V̂_new[k] ← V_k[π̂[k]]
        δ ← max(δ, |V̂_k'[k] − V̂_k[k]|)
    end for
    V̂_k ← V̂_new, N++;
end while
return [V̂_k[: −1], π̂[: −1]]
```

B. Q-learning

The parameters α, γ, ε, and the number of epochs are initialized with default values, obtained through experimental tuning, as explained in Section VI-A. A list for the layout, and a boolean indicating circular gameplay are the only required inputs for the algorithm.

To ensure convergence, both the learning rate α_t and the exploration rate ε_t gradually decay over time (see lines 12 and 13 in Algorithm 2).

Algorithm 2 Q-learning

Require: $layout$, $circle$, α, γ, ε, $epochs$
Ensure: Estimated $\hat{Q}$ and $\hat{\pi}$

```
1:  ∀k ∈ S, a ∈ D : Q̂(k, a) ← 0
2:  α_t ← α, ε_t ← ε
3:  for epoch ← 1 to epochs do
4:      k ← k_start
5:      while k ≠ k_final do
6:          a ← ChooseAction(k, ε_t, Q̂)                ▷ Eq. (3)
7:          T_k ← {(p(k'|t), φ_k') | t = (k, a)}
8:          r_k ← 1 + Σ_{t∈T_k} (φ_k' + p(k'|t) max_a' Q̂(k', a'))
9:          Temporal-Difference()                       ▷ Eq. (2)
10:         k ← k'
11:     end while
12:     α_t ← max(0.01, α / (1 + epoch/2000))
13:     ε_t ← max(0.01, ε e^(−epoch/7000))
14: end for
15: V̂_k ← min_a Q̂(k, a),   π̂_k ← arg min_a Q̂(k, a)
16: return [V̂_k[: −1], π̂_k[: −1]]
```

C. Snakes and Ladders Game

A high-level overview of the architecture is provided in Figure 19 (Appendix A).

V. Methodology

For consistency and reproducibility, all tests were conducted on an *Apple Silicon M2* chip with 16 GB of RAM. Each configuration was tested multiple times, and average values were used to ensure statistical reliability.

A. Strategies

A strategy defines the player's behavior throughout the game by determining which die to select based on the current state. Each strategy gives rise to a policy, which maps game states to dice choices.

We implemented six distinct strategies: four are sub-optimal (i.e. *AlwaysSecurity*, *AlwaysNormal*, *AlwaysRisky*, and *Random*), and two are optimal (i.e. *MDP*, and *Q-learning*).

Sub-optimal strategies do not adapt to the game context. Die are either selected randomly or remain fixed throughout the game, without regard to the player's current position.

The goal is to evaluate how different levels of decision-making sophistication impact player performance. By comparing fixed and random strategies (*sub-optimal*) with adaptive strategies derived from learning algorithms (*optimal*), we can quantify the benefits of using model-based (*MDP*) and model-free (*Q-learning*) approaches in this stochastic game environment.

B. Layouts

To understand how the layout could influence the policy obtained, we employed different configurations.

We intentionally avoided combining penalty and bonus traps in the same layout during initial testing to isolate the impact of each trap type on strategy performance. Mixing mechanics could make it difficult to determine whether results stem from interactions or individual trap effects. By keeping layouts mutually exclusive (i.e. represented in Figures 4, 7, 10, and 13), we ensure clear relationships between specific trap configurations and strategy effectiveness.

This controlled approach establishes baseline performance metrics, which serves as a foundation for exploring hybrid layouts, as in Figure 16, and possible future work.

C. Simulation

The simulation runs each board layout across all defined strategies. The Algorithm 3 (with more details in Appendix A) illustrates the process of a single simulation, for which the main goal is to simulate the progression from a given starting position to the final square, using a specified strategy. Applying it thousands of times for each position on the board gathers comprehensive policy results. This procedure is repeated for each layout defined earlier, and the results are presented in Section VI.

The comprehensive simulation framework, enhanced with advanced analytical instrumentation, captures the following key metrics for each strategy-layout combination:

Performance Metrics: This includes the average expected steps to completion (referred to as *Expectation*) and computational resource usage, including execution time and memory footprint.

Strategy Insights: This covers position-specific action probabilities, including the probabilities for the Security Die, Normal Die, and Risky Die at each of the 14 positions. Additionally, the position visit distribution is provided as *Expectations* for each position.

Game Configuration: The board layout is recorded, along with a termination mode flag that indicates whether the game follows a *circle* termination rule (requiring exact landing on the final position).

VI. Results and Analysis

A. Q-learning Hyper-parameters Tuning

To improve the performance of *Q-learning*, we conducted a hyper-parameter search by varying the learning rate α, discount factor γ, and exploration rate ε. Each combination was evaluated by running multiple simulations and computing the average reward obtained.

Figure 2 visualizes the impact of these parameters on performance. The best-performing models were identified based on their average reward. Higher rewards indicate more effective learning and better action selection throughout the game.

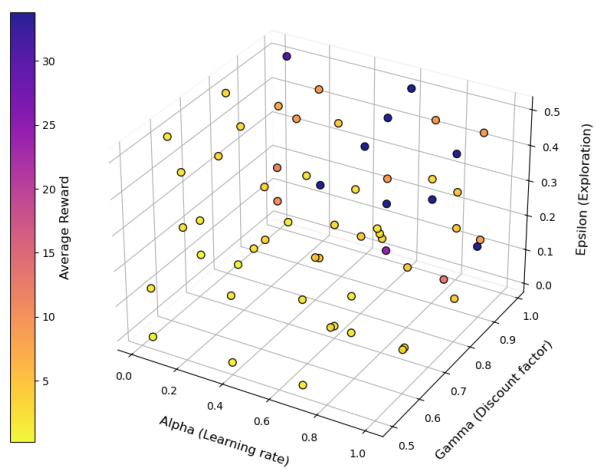

Fig. 2: Impact of hyper-parameters on average reward in Q-learning

Our results suggest that performance is highly sensitive to both α and ε. Too little exploration or an overly aggressive learning rate often led to sub-optimal convergence.

The optimal (i.e. more balanced) configuration (i.e. $\alpha = 0.5, \gamma = 0.99, \varepsilon = 0.35$) is used throughout the rest of this report.

B. Memory and runtime

In terms of execution time and memory usage, the observations shown in Figure 3 remain consistent regardless of

the layout. *Q-learning* consistently requires more time and memory than *MDP*. This can be attributed to several factors, including the size of the Q-table, the iterative updating of Q-values, and the ε-greedy exploration strategy, all of which contribute to higher computational demands.

Notably, the game board design does not affect these observations. Whether the layout includes a circular design or not, the performance trend remains unchanged: *Q-learning* systematically consumes more resources than *MDP*.

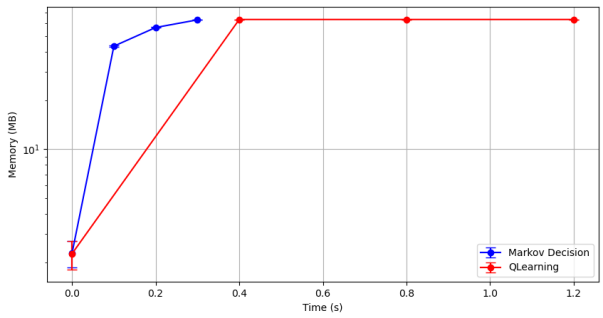

Fig. 3: *prison_only* layout: Execution Time and Memory Usage (w/ circle)

C. Bonus only

We first analyze the *bonus_only* layout (Figure 4) on a game board that does not follow a circular design. The optimal strategies exhibit an average expected value close to 1 for most game cells (Figure 5), independently of the used method.

The *always_security* strategy performs the worst in this layout. Since the player can only advance by one cell or remain stationary, and bonuses cannot be triggered, significantly slowing down progress.

The *always_normal* and *random* strategies yield similar results across all cells, with average expectations much lower than those of the *always_security* strategy.

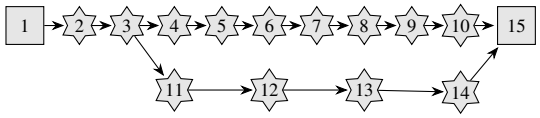

Fig. 4: *bonus_only* layout

On the other hand, the *always_risky* strategy prioritizes advancing by more steps at every opportunity, providing a considerable advantage in this layout. As expected, when comparing *always_risky* with *MDP* and *Q-learning*, their values are identical. This indicates that optimization methods consistently selects the risky die in this configuration, as it focuses on advancing by the maximum number of steps possible.

When introducing a circular design (Figure 6), the overall trends remain similar. *MDP*, *Q-learning*, and *always_risky* still converge to the same expected value, confirming that the risky die remains the optimal choice. Similarly, *always_normal* and *random* strategies continue to produce identical outcomes, while *always_security* remains the least effective approach.

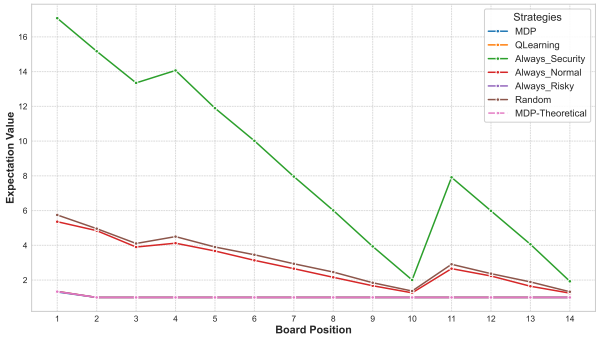

Fig. 5: Strategies expectations for *bonus_only* layout w/o circle

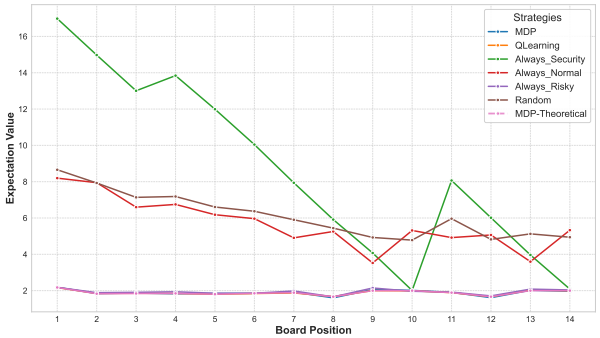

Fig. 6: Strategies expectations for *bonus_only* layout w/ circle

D. Prison Trap Only

For the layout illustrated in Figure 7, all strategies except *always_security* converge toward the same value (Figure 8). Due to the presence of prison traps, the player must skip a turn before resuming movement. However, since triggering the effect with a move of 2 or more can offset the skipped turn, it makes it more efficient than moving by 1 twice without penalties. *MDP* and *Q-learning* offer as expected, the best strategy.

As in the previous layout, the *always_security* strategy performs the worst for the same reasons — it severely limits movement and prevents efficient progress.

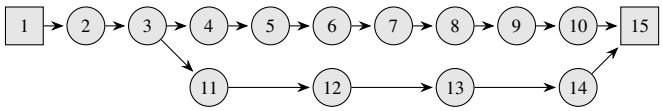

Fig. 7: *prison_only* layout

With a circular design, the results (Figure 9) change significantly. The optimal strategies demonstrate a clear advantage, with *MDP* and *Q-learning* converging. Unlike the previous layout, the optimal strategy involves a combination of different die rather than relying only on a single one.

Interestingly, the worst-performing strategy is no longer *always_security* but *always_risky*. A plausible explanation for this is that the probability of landing precisely on the final square without overshooting is very low. Additionally, the guaranteed activation of traps at every step significantly degrades the performance of this strategy, making it even less viable in this layout.

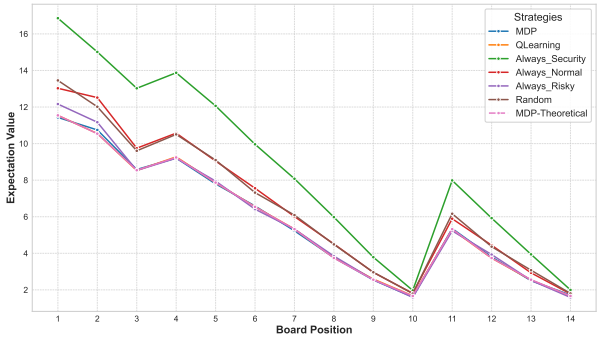

Fig. 8: Strategies expectations for *prison_only* layout w/o circle

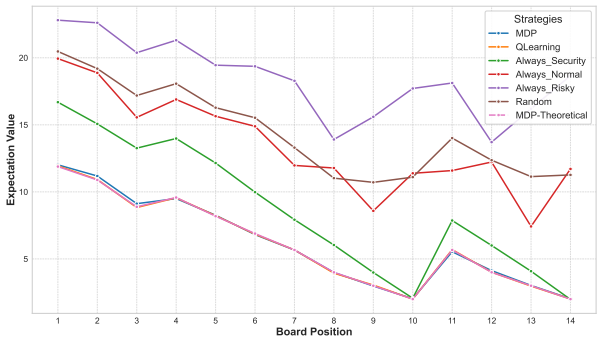

Fig. 9: Strategies expectations for *prison_only* layout w/ circle

E. Penalty Trap Only

For the layout illustrated in Figure 10, without a circular design, the optimal strategies perform best, with their values clearly converging. When comparing these two strategies with *always_security*, we observe very similar results (Figure 11). This suggests the vast majority of chosen die for the optimal strategies to be *security* dice.

This strategy is more challenging for *Q-learning* (from 1000 to 10,000 simulations), requiring more simulations to approach *MDP* performance. The layout induces frequent backtracking, leading to redundant exploration and slower convergence.

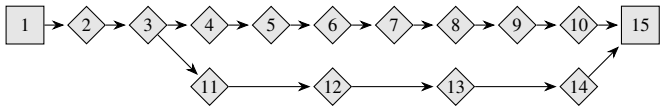

Fig. 10: *penalty_only* layout

Among the sub-optimal strategies, *always_risky* stands out with particularly high values. In most of the positions, reaching the final square is impossible. This is because advancing when triggering an effect is only possible by rolling a 4, which does not exist. As a result, only by rolling without triggering an effect can a player mover forward, for which the probability is small for *always_risky*.

Introducing a circular design into the layout exacerbates the challenges for sub-optimal strategies. The Figure 12 clearly shows an overall increase in average expectations. Reaching the final square with the *always_risky* strategy becomes even less feasible than before. While *MDP* and *always_security* still converge, *Q-learning* does not behave as expected. It deviates

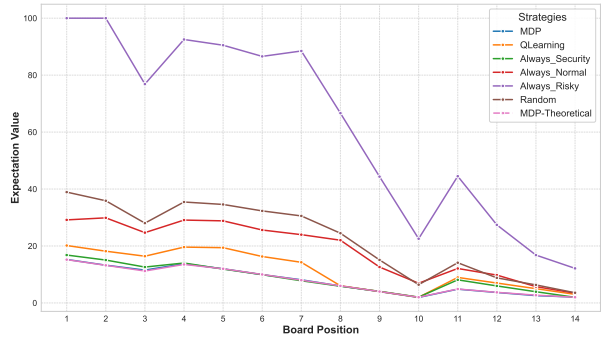

Fig. 11: Strategies expectations for *penalty_only* layout w/o circle

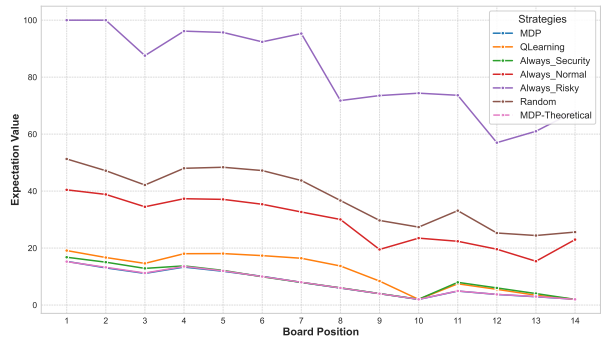

Fig. 12: Strategies expectations for *penalty_only* layout w/ circle

from the expected values like for *MDP*. This instability may be due to suboptimal hyper-parameter tuning, or still an insufficient amount of simulations.

F. Restart Trap Only

For the layout illustrated in Figure 13, the *MDP* and *always_security* strategies yield identical values (Figure 14), indicating that the security die is the most effective choice in this scenario. In contrast, *Q-learning* produces highly inconsistent results. This is explained by the nature of the layout, which is even more challenging than the previous one, in Section VI-E. It produces extreme backtracking, leading to high redundant exploration, hence the very slow convergence (even with 10,000 iteration). Better hyper-parameters were used to mitigate this situation (i.e. $\alpha = 0.05, \gamma = 0.95, \epsilon = 0.4$) with much less exploration and much more exploitation.

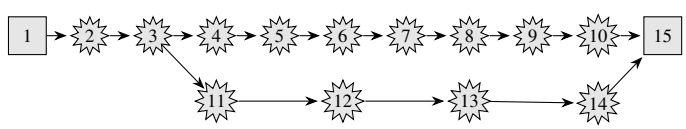

Fig. 13: *restart_only* layout

For the other strategies, the average expectation is significantly high, as each triggered trap sends the player back to the starting square.

With a circular design, the overall trend (Figure 15) remains similar, but is exacerbated as in Section VI-E, and for the same reasons.

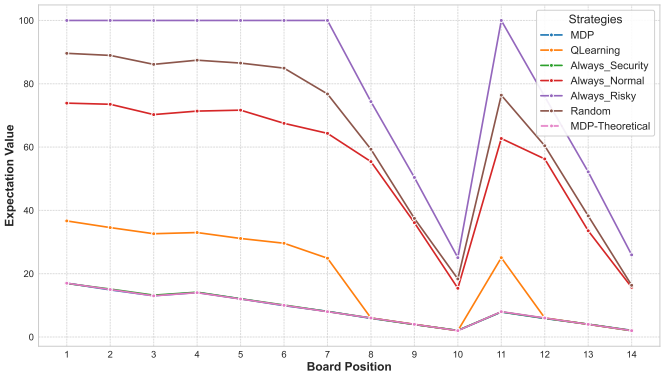

Fig. 14: Strategies expectations for *restart_only* layout w/o circle

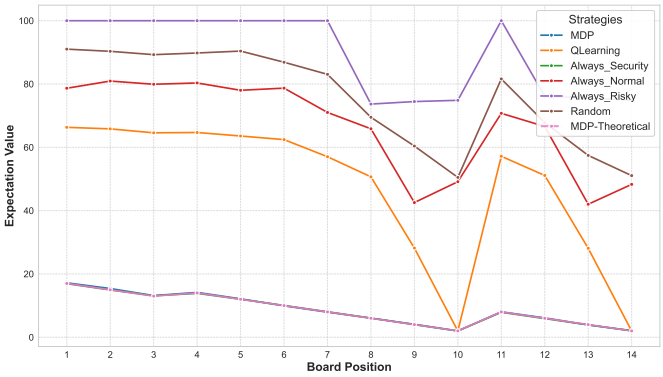

Fig. 15: Strategies expectations for *restart_only* layout w/ circle

G. Trap combination

This punitive layout illustrated in Figure 16 contains all possible traps. The *MDP* and *Q-learning* are the best strategies with *always_security* without a circular design. Their values (Figure 18) converge for each position. We can deduce from this that it is the security die that is used most often. We note that the *Q-learning* do not converge with a circular design but it is close, suggesting the need for better hyper-parameters, or more simulations. As for the others, the high expectations prove that they are sub-optimal for this layout.

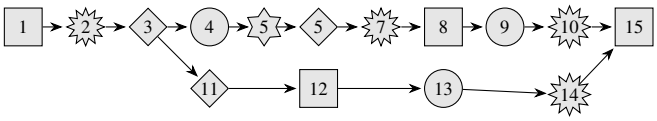

Fig. 16: Layout with a combination of traps

In a design without a circle, the trend (Figure 17) stay the same, except for the sub-optimal strategies, which are different from *always_security*. In fact, the values are of the same order and are very close. On the other hand, for the other sub-optimal strategies, the values are better for this design but still perform less well.

VII. Conclusion

This report investigated the use of decision-making algorithms for optimal die selection in a stochastic game environment. We demonstrated the effectiveness of adaptive approaches based on *Markov Decision Processes (MDP)* and *Q-learning* by comparing them with sub-optimal strategies.

MDP consistently yielded stable and optimal results across all layouts, in comparison with sub-optimal strategies which are only simpler. In contrast, *Q-learning* showed promise but suffered from convergence issues. The difficulty of a layout for *Q-learning* can often be inferred from its trap composition: layouts with many *penalty* or especially *restart* cells tend to hinder convergence due to frequent backtracking, whereas *bonus* cells generally improve learning efficiency. These findings suggest that a single hyper-parameter configuration may not generalize well across all layout types, but that quantifying the layout difficulty should be possible.

Overall, this work underscores the value of state-aware policies and provides a foundation for future research on hybrid layouts, *Q-learning* robustness, and adaptive strategy learning.

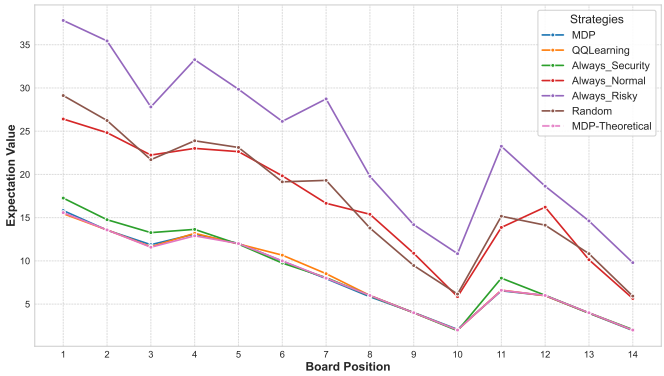

Fig. 17: Strategies expectations for *combination* layout w/o circle

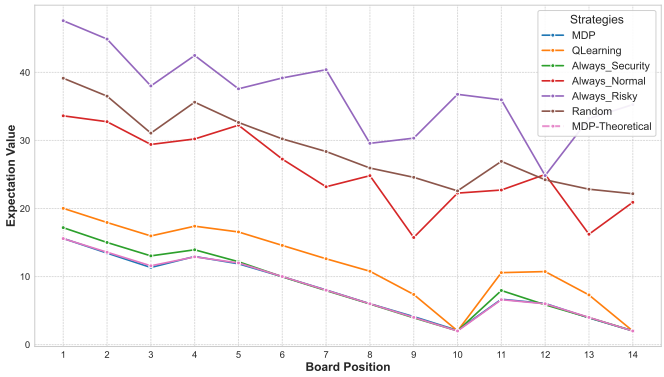

Fig. 18: Strategies expectations for *combination* layout w/ circle

References

[1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Global ed. Hoboken, NJ, USA: Pearson, 2020, pp. 552–562, Sections 16.1–16.2.

[2] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray, *Algorithms for Decision Making*, Cambridge, MA, USA: MIT Press, 2022, pp. 133–139, Sections 7.1–7.8.

[3] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Global ed. Hoboken, NJ, USA: Pearson, 2020, pp. 840–865, Chapter 23.

[4] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray, *Algorithms for Decision Making*, Cambridge, MA, USA: MIT Press, 2022, pp. 199–215, Chapter 17.

APPENDIX A

I. Project Structure

The game is implemented using an object-oriented approach, which UML is described in Figure 19, structured around a central `BoardGame` class that encapsulates the board layout, game states, and available dice.

Each cell on the board is represented by a `State` object, characterized by a position and a `CellType`. Different dice behaviors are modeled in the `Die` class, which defines possible moves, associated probabilities, and the likelihood of triggering traps or bonuses. Transitions and trap-handling are managed by the `TransitionManager`, which determines state transitions based on dice rolls.

The `DiceStrategy` class implements various decision-making strategies, including optimal policies derived from *Value Iteration MDP* and *Q-learning*.

The overall simulation framework, including strategy comparison and empirical evaluation, is orchestrated by the `BoardGameSimulator`. Additionally, dynamic board configurations can be generated using the `LayoutGenerator`.

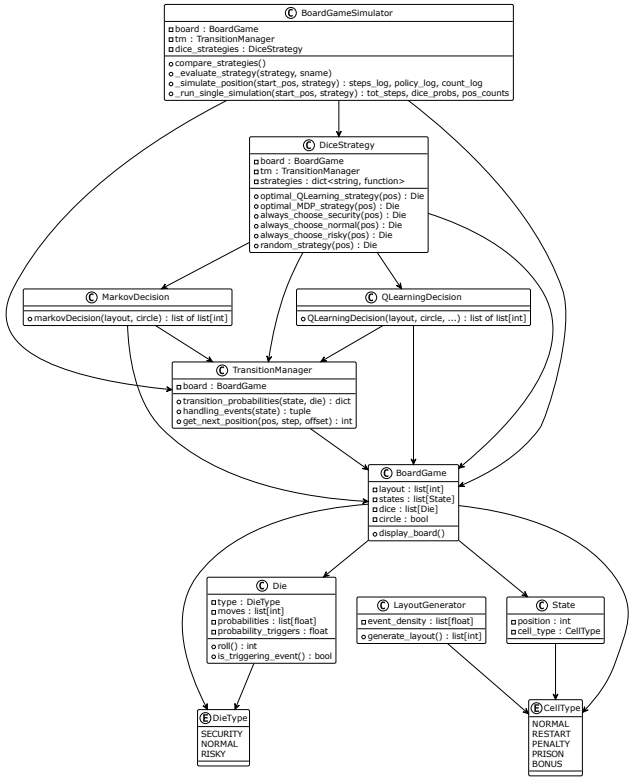

Fig. 19: UML diagram representing the class structure and interactions within the project implementation.

II. Simulation Implementation

Algorithm 3 simulates a single game from a given position using a chosen strategy. It is repeated thousands of times per position and layout to derive policy statistics.

In each iteration, the current position is retrieved, and the strategy function is called to select a dice. The chosen dice is rolled to determine the number of steps to move. The function then updates the tracking arrays: *position_counts* is incremented for the current position, and *dice_choices* is updated based on the selected dice type.

Algorithm 3 Single Game Simulation

Require: start_pos, strategy
Ensure: tot_steps, dice_probs, position_counts

```
function SIMULATE(start_pos, strategy)
    position_counts ← array of zeros (size 14)
    dice_choices ← array of zeros (3×14)
    current_state ← board.states[start_pos]
    curr_pos ← current_state.position
    tot_steps ← 0, max_step ← 100
    while curr_pos≠last_cell and tot_steps<max_step do
        curr_pos ← current_state.position
        die ← APPLY_STRATEGY(curr_pos)
        move ← ROLL(die)
        position_counts[curr_pos]++
        dice_choices[die.type.value-1, curr_pos]++
        offset ← GET_OFFSET
        new_pos ← NEXT_POS(curr_pos, move, offset)
        state ← board.states[new_pos]
        if die triggers event then
            (pos, penalty) ← HANDLE_EVENTS(state)
            total_steps += penalty
            new_state ← board.states[pos]
        else
            new_state ← state
        end if
        current_state ← new_state
        curr_pos ← current_state.position
        tot_steps++
    end while
    dice_probs ← dice_choices / position_counts
    Replace NaN/∞ with 0 in dice_probs
    return tot_steps, dice_probs, position_counts
end function
```

This function also calculates the movement parameters, including potential offsets when the player enters the *slow lane*. Special events triggered by the die are handled by calling the *handling_events* function, which updates the position and accounts for any penalties (such as moving backward or skipping a turn). The penalty is added to the total step count, and the player's state is updated accordingly. The simulation loop continues until the player reaches the final position or the total number of steps exceeds the predefined maximum (*max_steps*).

After the loop ends, the function calculates the probabilities of selecting each type of die at every position. Finally, it returns three key metrics: the total number of steps taken (*total_steps*), the dice selection probabilities (*dice_probs*), and the position visit counts (*position_counts*).